

ELO329 - Diseño y Programación Orientados a Objetos

Memoria dinámica en C++

Agustín J. González
Patricio Olivares R.

Universidad Técnica Federico Santa María

Propósito de la clase

En C++ un objeto puede vivir como variable local, como atributo de otro objeto o en memoria dinámica.

Esta clase se concentra en el tercer caso: objetos creados en el heap con `new`.

El objetivo no es solo aprender la sintaxis, sino entender la responsabilidad que aparece cuando una clase administra memoria.

En C++, quien adquiere un recurso debe tener claro cuándo y cómo lo libera.

Contenidos

1. Stack y heap
2. `new` y `delete`
3. Fugas de memoria
4. Punteros inválidos
5. Arreglos y punteros
6. Arreglos dinámicos
7. Punteros como atributos
8. Constructor copia y asignación
9. Regla de tres
10. Contenedores como alternativa

Recordatorio: duración de objetos

Ubicación	Ejemplo	Quién termina la vida
Stack	<code>Student s;</code>	El bloque donde se declaró
Heap	<code>new Student</code>	El programador, usando <code>delete</code>
Datos estáticos	<code>static int count;</code>	El programa al terminar

La memoria dinámica sirve cuando la vida del objeto no coincide con el bloque donde se creó.

Crear un objeto en el heap

```
int *p = new int;  
  
*p = 25;  
std::cout << *p << std::endl;  
  
delete p;
```

Lectura:

- `new int` crea un entero en el heap.
- `p` guarda la dirección de ese entero.
- `delete p` libera el entero.

Qué hace new

`new` realiza dos acciones:

1. Reserva memoria suficiente para el objeto.
2. Construye el objeto en esa memoria.

```
Student *ps = new Student;
```

En este caso se invoca el constructor de `Student`.

El resultado de `new` es una dirección, por eso se guarda en un puntero.

Qué hace delete

`delete` también realiza dos acciones:

1. Invoca el destructor del objeto.
2. Devuelve la memoria al heap.

```
delete ps;
```

Después de esto, el objeto ya no existe.

El puntero `ps` conserva una dirección, pero esa dirección ya no debe usarse.

Comparación con Java

En Java, los objetos se crean con `new` y el recolector de basura decide cuándo liberar memoria.

En C++, `new` no implica recolección automática.

```
Student *ps = new Student;  
  
// ...  
  
delete ps;
```

Si el programa usa `new`, debe haber una estrategia clara para liberar.

new dentro de una función

```
void f() {  
    Student *ps = new Student;  
  
    // usar ps  
  
    delete ps;  
}
```

Cuando la función termina, `ps` deja de existir por ser variable local.

Pero el objeto creado con `new` no desaparece por eso.

Por eso debe llamarse `delete` antes de perder la dirección.

Fuga de memoria

```
void f() {  
    Student *ps = new Student;  
  
    // usar ps  
} // ps desaparece, el Student queda en el heap
```

Esto es una fuga de memoria.

El programa ya no tiene una dirección para liberar el objeto.

Si `f` se llama muchas veces, la memoria usada puede crecer sin control.

Retornar un objeto del heap

A veces la función crea un objeto y transfiere su dirección.

```
Student *makeStudent() {  
    Student *ps = new Student;  
    return ps;  
}
```

Quien llama recibe la responsabilidad.

```
Student *s = makeStudent();  
  
// usar s  
  
delete s;
```

Propiedad del recurso

Cuando una función retorna un puntero creado con `new`, debe quedar claro quién lo libera.

```
makeStudent() crea  
main() recibe  
main() elimina
```

Si esa regla no está documentada o encapsulada, el código se vuelve frágil.

La memoria dinámica no falla solo por sintaxis. Falla cuando no hay una política clara de propiedad.

Punteros inválidos

```
double *pd = new double;  
*pd = 3.523;  
  
delete pd;  
  
*pd = 4.2; // error: el objeto ya no existe
```

Después de `delete`, usar `*pd` es comportamiento indefinido.

El programa puede fallar inmediatamente, parecer funcionar o corromper datos.

Buena práctica mínima

Después de liberar, puede asignarse `nullptr`.

```
delete pd;  
pd = nullptr;
```

Esto no arregla otros punteros que apunten al mismo objeto, pero evita reutilizar accidentalmente este puntero.

```
if (pd != nullptr) {  
    *pd = 4.2;  
}
```

Arreglos y punteros

El nombre de un arreglo puede usarse como dirección del primer elemento.

```
int scores[50];  
int *p = scores;  
  
*p = 99;  
std::cout << scores[0]; // 99
```

p puede avanzar.

```
p++; // ahora apunta a scores[1]
```

El nombre del arreglo no es reasignable

```
int scores[50];  
int *p = scores;  
  
p++;      // ok  
scores++; // error
```

`scores` se comporta como una dirección fija al primer elemento.

`p`, en cambio, es una variable puntero independiente.

Arreglos de punteros

Un arreglo puede guardar direcciones a objetos creados en el heap.

```
Student *elo329[10];

for (int i = 0; i < 10; i++) {
    elo329[i] = new Student;
}

for (int i = 0; i < 10; i++) {
    delete elo329[i];
}
```

El arreglo está en un lugar; los objetos apuntados pueden estar dispersos en el heap.

Arreglo completo en el heap

```
double *samples = new double[10];  
samples[0] = 36.2;  
delete [] samples;
```

Para arreglos creados con `new[]`, se debe usar `delete[]`.

No se escribe el tamaño al liberar.

Emparejar creación y liberación

Creación	Liberación correcta
<code>new T</code>	<code>delete p</code>
<code>new T[n]</code>	<code>delete [] p</code>

Mezclar estas formas es un error.

```
Student *s = new Student;  
delete [] s; // incorrecto
```

El operador usado al liberar debe corresponder al operador usado al crear.

Punteros como atributos

Los punteros son útiles cuando una clase administra un recurso.

```
class Student {  
public:  
    Student();  
    ~Student();  
  
private:  
    std::string *courses;  
    int count;  
};
```

La clase puede ocultar `new` y `delete` detrás de su constructor y destructor.

Constructor y destructor

```
Student::Student() {  
    courses = new std::string[50];  
    count = 0;  
}  
  
Student::~~Student() {  
    delete [] courses;  
}
```

La idea es correcta: el objeto adquiere el recurso al construirse y lo libera al destruirse. Pero ahora aparece un problema nuevo: copiar el objeto.

Copia baja por defecto

Si una clase tiene un puntero como atributo, la copia por defecto copia la dirección.

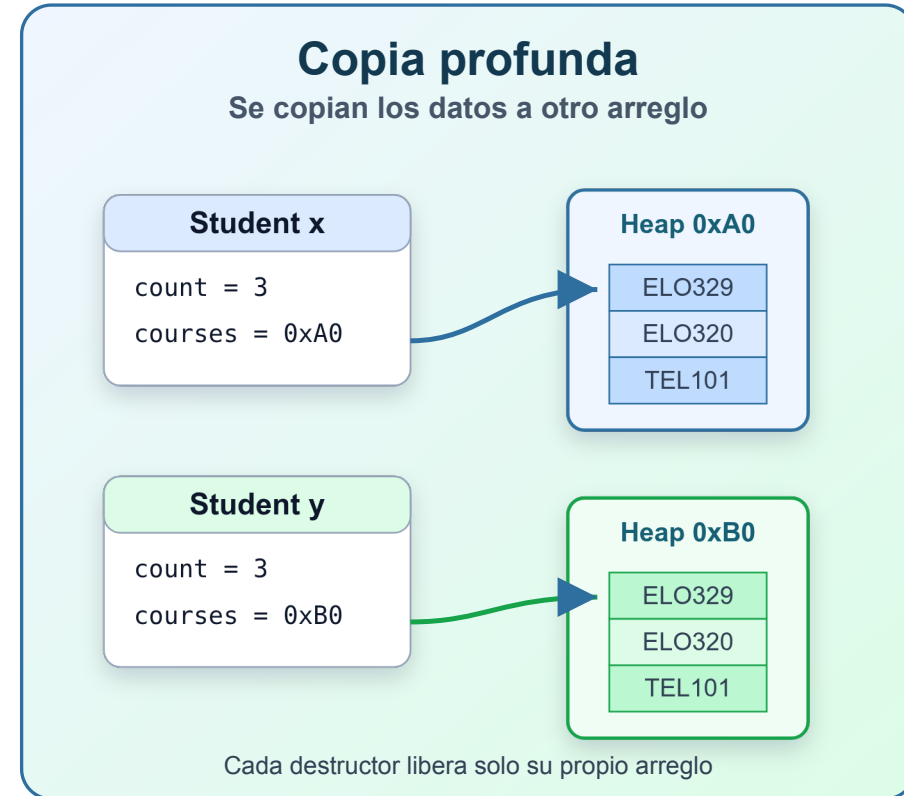
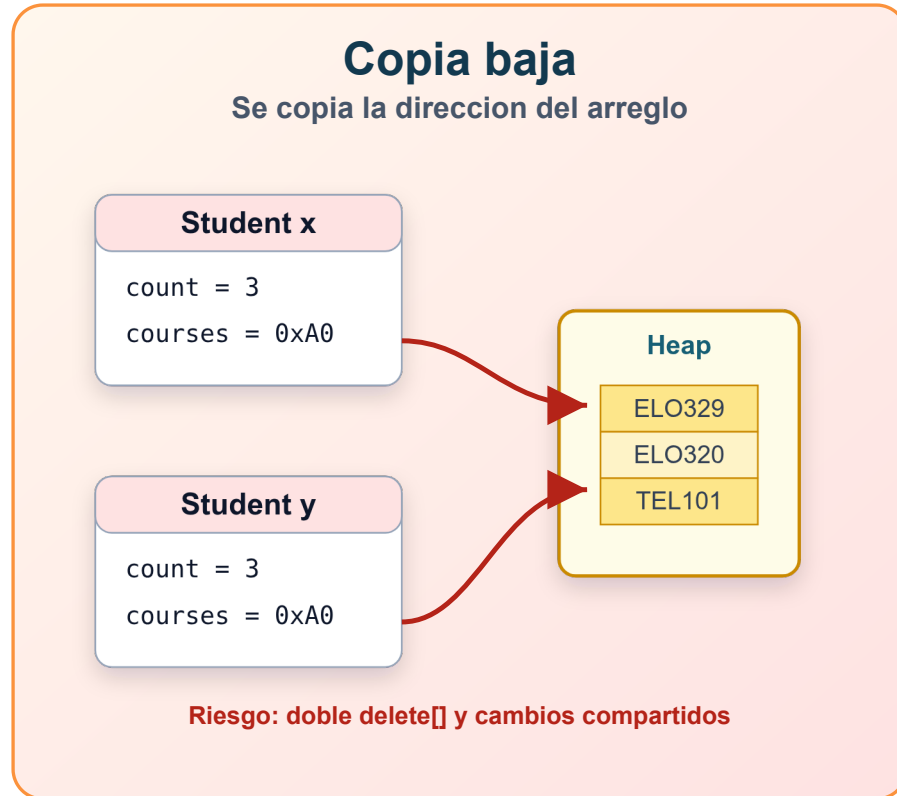
```
Student x;  
Student y(x);
```

Después de esto, `x.courses` e `y.courses` pueden apuntar al mismo arreglo.

Eso no crea dos listas de cursos.

Crea dos objetos que comparten el mismo recurso sin haberlo decidido explícitamente.

Copia baja versus copia profunda



La copia baja comparte el puntero.

La copia profunda crea un nuevo arreglo y copia los elementos.

Problemas de la copia baja

Si dos objetos comparten el mismo arreglo dinámico sin coordinación:

- Cambiar cursos en un estudiante afecta al otro.
- Al destruirse ambos, ambos intentan liberar la misma memoria.
- El segundo `delete[]` produce comportamiento indefinido.

Este es el motivo por el que el constructor copia por defecto no basta en clases que poseen recursos.

Constructor copia correcto

```
Student::Student(const Student &other) {  
    count = other.count;  
    courses = new std::string[count];  
  
    for (int i = 0; i < count; i++) {  
        courses[i] = other.courses[i];  
    }  
}
```

Ahora cada objeto tiene su propio arreglo.

La copia conserva los valores, no la propiedad del mismo puntero.

Paso por valor también copia

```
void foo(Student s) {  
    // ...  
}  
  
Student jose;  
foo(jose);
```

La variable local `s` se construye usando el constructor copia.

No se ejecuta una asignación después.

Por eso el constructor copia debe ser correcto incluso si el código no llama explícitamente `Student copia(jose)`.

Operador de asignación

También debemos manejar este caso:

```
Student a;  
Student b;  
  
b = a;
```

La asignación ocurre entre objetos que ya existen.

No es lo mismo que construir `b` a partir de `a`.

Sobrecarga de operator=

```
Student &Student::operator=(const Student &other) {  
    if (this == &other) {  
        return *this;  
    }  
  
    delete [] courses;  
    count = other.count;  
    courses = new std::string[count];  
  
    for (int i = 0; i < count; i++) {  
        courses[i] = other.courses[i];  
    }  
  
    return *this;  
}
```

La asignación debe liberar el recurso antiguo y crear una copia propia del recurso nuevo.

Por qué retornar Student&

El operador de asignación retorna una referencia al objeto izquierdo.

```
return *this;
```

Eso permite encadenar asignaciones.

```
a = b = c;
```

También coincide con el comportamiento esperado de los tipos incorporados de C++.

Regla de tres

Si una clase necesita escribir manualmente alguno de estos tres:

- Destructor
- Constructor copia
- Operador de asignación

probablemente necesita escribir los tres.

Esta regla aparece porque los tres participan en la misma responsabilidad: administrar la vida de un recurso.

Mejor alternativa: contenedores

Muchas veces no necesitamos administrar memoria manualmente.

```
class Student {  
public:  
    Student() = default;  
  
private:  
    std::vector<std::string> courses;  
};
```

`std::vector` ya implementa constructor copia, asignación y destructor correctamente.

La clase queda más simple y menos propensa a errores.

Nota de C++ moderno

En código C++ moderno, `new` y `delete` aparecen menos en código de aplicación.

Se prefieren:

- Objetos automáticos cuando la vida coincide con el bloque.
- Contenedores estándar como `std::vector` y `std::string`.
- Punteros inteligentes cuando debe existir propiedad dinámica.

En este curso estudiamos `new` y `delete` porque explican qué problema resuelven esas abstracciones.

Conexión con clases anteriores

Tema anterior	Relación con memoria dinámica
Constructor	Adquiere recursos
Destructor	Libera recursos
<code>const T &</code>	Evita copias innecesarias
Constructor copia	Define cómo duplicar un objeto
<code>this</code>	Permite detectar autoasignación

La administración de memoria no es un tema aislado: afecta directamente el diseño de clases.

Ideas clave

- `new` crea objetos en el heap y retorna una dirección.
- `delete` destruye el objeto y libera su memoria.
- Toda memoria creada con `new` debe tener una política clara de liberación.
- Usar memoria después de `delete` produce comportamiento indefinido.
- `new[]` debe emparejarse con `delete[]`.
- Si una clase posee un puntero, la copia por defecto suele ser peligrosa.
- La copia profunda evita compartir accidentalmente recursos.
- Si se implementa destructor, constructor copia u operador `=`, revisar la regla de tres.
- Preferir contenedores estándar cuando resuelven el problema.